Code Coverage Analysis in C using GCOV and GCOVR

1. Introduction

In safety-critical and high-reliability systems such as **defense systems, aerospace software, automotive ECUs, and embedded firmware**, testing is not just about checking if the program runs successfully. It is also important to verify whether **every logical path in the code has been executed during testing**.

This is where **code coverage analysis** becomes essential.

Code coverage tools analyze the execution of a program and determine:

- Which lines of code were executed
- Which branches or decision paths were taken
- Which functions were invoked
- Which parts of the code were never tested

Coverage analysis helps developers identify **untested code paths**, which could potentially hide bugs or incorrect logic.

For example, consider a radar-based defense decision system where a program decides whether to:

- Ignore a distant target
- Monitor a nearby target
- Track a high-speed object
- Launch countermeasures against a hostile target

If some of these decision paths are never executed during testing, the system may behave incorrectly in real-world scenarios.

Therefore, **code coverage analysis ensures that the test suite exercises as much of the program logic as possible.**

2. Types of Code Coverage

Code coverage tools typically measure multiple aspects of program execution.

Line Coverage

Indicates how many lines of source code were executed.

Example:

Lines executed: 94.74%

This means most lines were executed, but some lines were not tested.

Branch Coverage

Measures whether both **true and false outcomes of conditional statements** were executed.

Example condition:

if(speed > 300)

Branch coverage checks if both cases occurred:

- speed > 300
- speed <= 300

Function Coverage

Measures whether every function defined in the program was executed.

3. GCC Code Coverage Tools

The GNU toolchain provides a built-in coverage analysis mechanism through a set of related tools and generated files. These tools work together to collect and analyze execution data.

The primary components are:

Tool	Purpose
gcov	Generates coverage statistics from execution data
gcno	Stores static program structure
gcda	Stores runtime execution data

These files are generated automatically when the program is compiled with coverage instrumentation.

4. Coverage Data Files

4.1 GCNO File

GCNO stands for GNU Coverage Notes Object file.

This file is generated during the compilation phase when coverage instrumentation is enabled using the GCC option:

The .gcno file contains static information about the program structure, including:

- control flow graph
- basic blocks
- branch relationships
- mapping between source lines and compiled instructions

4.2. GCDA File

GCDA stands for **GNU Coverage Data** file.

This file is generated during **program execution**.

When the program runs, the instrumentation inserted by the compiler records runtime statistics and stores them in the .gcda file.

The .gcda file contains dynamic execution information such as:

- how many times each line was executed
- how many times each branch was taken
- function invocation counts

4.3. GCOV

GCOV stands for GNU Coverage Tool.

It is a utility provided by the GNU Compiler Collection (GCC) used to analyze the execution of a program and determine how much of the source code was exercised during testing.

GCOV processes runtime data collected from instrumented programs and produces a detailed report showing:

- execution count of each line of code
- branch execution information
- function execution statistics

5. **Example Program (radar.c)**

```c
#include <stdio.h>

typedef struct {
    int distance;   // km
    int speed;      // m/s
    int isHostile;  // 0 = unknown, 1 = hostile
} Target;

void evaluateTarget(Target t)
{
    if(t.distance > 200)
    {
        printf("Target too far. Ignoring.\n");
    }
    else
    {
        printf("Target within monitoring range.\n");

        if(t.speed > 300)
        {
            printf("High speed object detected.\n");

            if(t.isHostile)
            {
                printf("Hostile target! Launch countermeasures.\n");
            }
            else
            {
                printf("Unknown high speed object. Track closely.\n");
            }
        }
        else
        {
            printf("Slow object. Continue surveillance.\n");
        }
    }
}

int main()
{
    Target t1 = {250, 200, 0};
    Target t2 = {150, 350, 0};
    Target t3 = {120, 400, 1};

    evaluateTarget(t1);
    evaluateTarget(t2);
    evaluateTarget(t3);

    return 0;
}
```

6. Running Coverage Analysis

The following steps demonstrate how to run coverage analysis using GCC and GCOV on Windows.

Step 1 – Compile with Coverage Instrumentation

```
gcc --coverage radar.c -o radar
```

This command performs two actions:

- Compiles the program
- Inserts coverage instrumentation

Output files created are - radar.exe , radar.gcno

Step 2 – Execute the Program

```
PS C:\Users\hemas\Desktop\Dhruval> ./radar
Target too far. Ignoring.
Target within monitoring range.
High speed object detected.
Unknown high speed object. Track closely.
Target within monitoring range.
High speed object detected.
Hostile target! Launch countermeasures.
```

Example program output -

Also, generates - radar.gcda

Step 3 – Run GCOV Analysis

```
PS C:\Users\hemas\Desktop\Dhruval> gcov radar.c
File 'radar.c'
Lines executed:94.74% of 19
Creating 'radar.c.gcov'
```

7. Visual Coverage Reports using GCOVR

While gcov produces text-based output, GCOVR converts coverage data into visual reports.

These reports can be generated in formats such as:

- **HTML**
- **XML**
- **JSON**
- **SonarQube format**

Installing GCOVR (Windows)

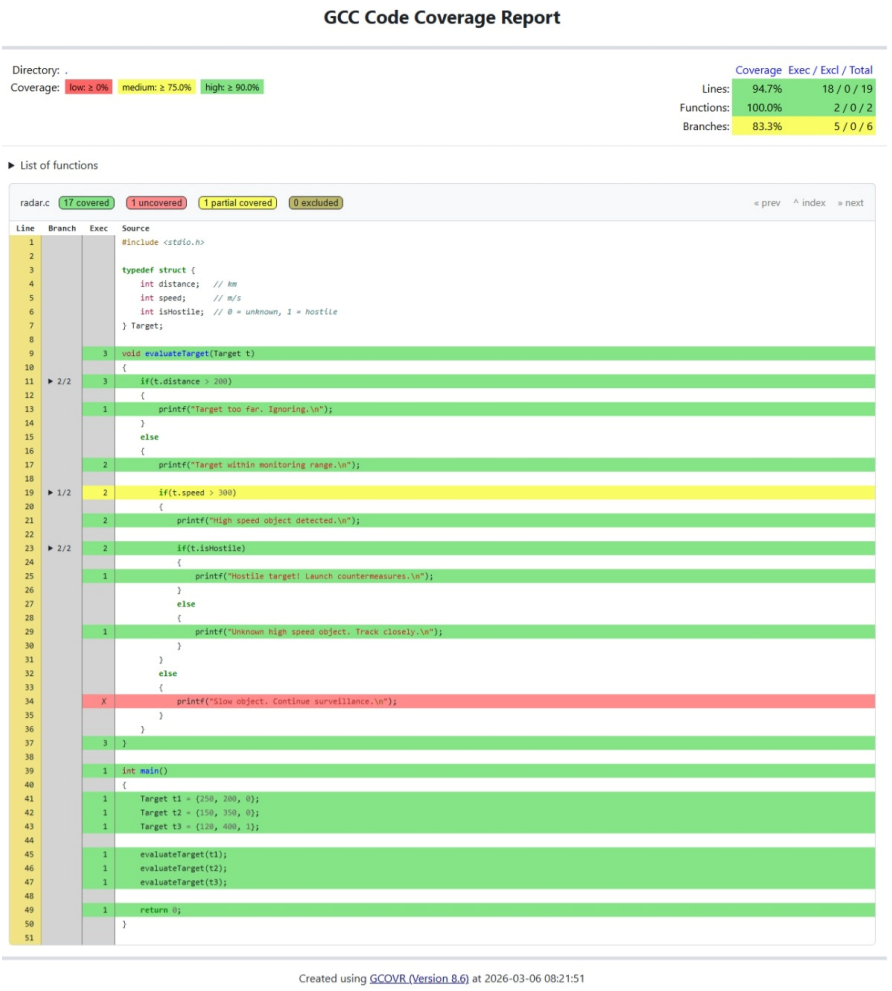
pip install --user gcovr

Generating HTML Coverage Report

python -m gcovr -r . --html --html-details -o coverage.html

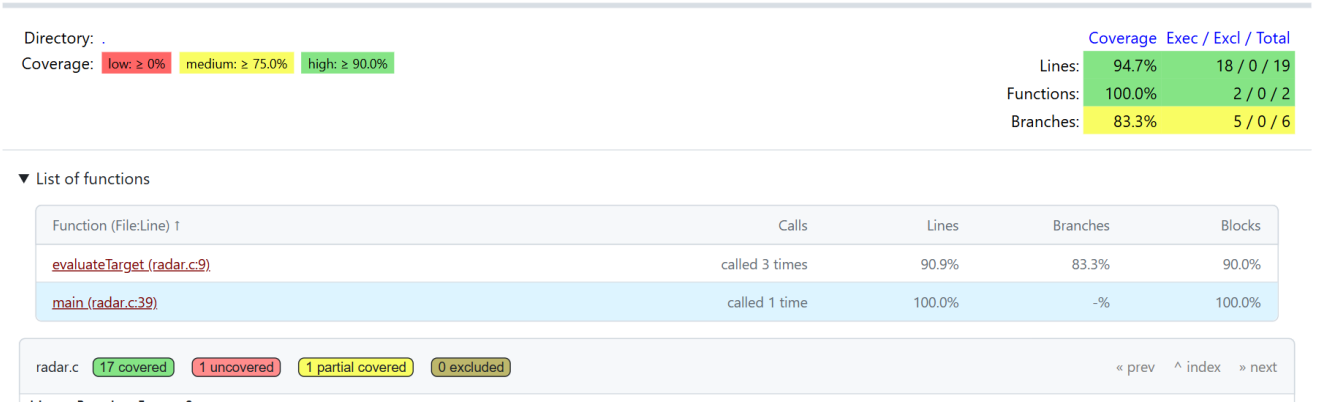
it will generate .html and .js files

8. Coverage Visualization



The HTML report highlights source code based on execution.

Color	Meaning
Green	Executed lines
Red	Unexecuted lines
Yellow	Partially covered branches



9. Key Takeaways

- Code coverage measures how much of the program logic is tested.
- GCC provides built-in coverage instrumentation using gcov.
- Compilation with --coverage generates .gcno files.
- Program execution generates .gcda files containing runtime data.
- gcov converts execution data into human-readable reports.
- Tools like GCOVR transform raw coverage data into visual HTML reports.
- Coverage reports highlight untested code paths, helping developers improve test suites